

ReStock AI

A Human-Governed Autonomous Purchasing Officer

Detailed Project Flow & Technical Documentation

Agentika 2.0 · NaSCon 2026 · Theme: Human + AI — Better Together

Field	Detail
Team	Chat GPTwo
Members	Chaudhary Muhammad Huzaifa Ali · Umamah Nasir Khan
Live deployment	https://alternate-restless-forest-3113-github-app.alternatefutures.ai
Source code	https://github.com/chhuzaifa928/Restocker
Stack	Python · FastAPI · Q-Learning · SQLite · Docker · MCP

1. Executive Summary

ReStock AI is an AI-driven stock-replenishment officer for a distributor. A pre-trained Q-Learning agent decides how much to reorder for each product; a safety layer called the dual gate then routes every decision one of three ways: it auto-executes the routine low-risk orders, escalates the expensive, low-confidence or unfamiliar ones to a human for one-click approval, and blocks the unsafe ones outright. An LLM writes a plain-English rationale for each escalation — it explains, it never decides.

Core principle: the AI handles the volume, the human owns the judgment. This directly embodies the event theme, Human + AI: Better Together.

2. The Problem

Distributors make thousands of restock decisions every week. The two naive approaches both fail:

- **Full automation** — a single mispriced or oversized order silently burns the budget.
- **Fully manual** — the team drowns in routine approvals and reacts too slowly to demand.

The routine volume needs automation; the risky calls need human judgment. ReStock AI splits the work along exactly that line.

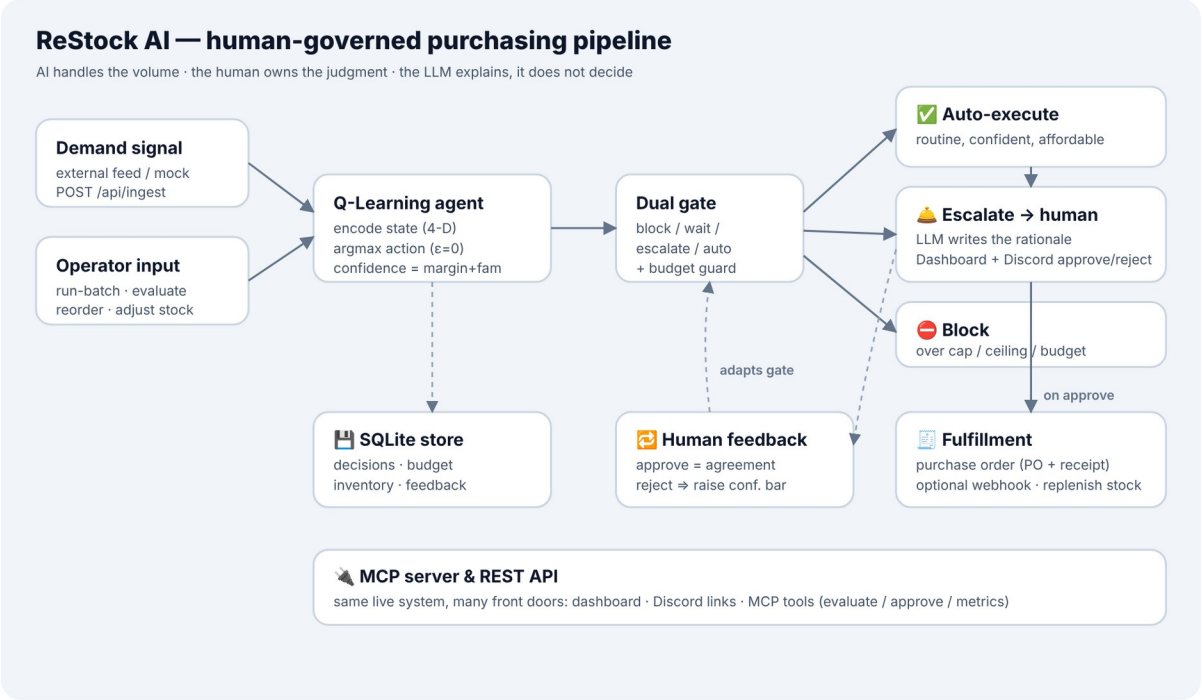
3. The Solution & Theme Alignment

The agent proposes; the gate governs; the human decides the exceptions. Each decision is fully auditable, and the system does real work — it issues purchase orders, notifies a human on Discord, and reacts to incoming demand signals.

Route	When	What happens
Auto-execute	Routine, confident, affordable	Order runs automatically; stock replenished; PO issued
Escalate	Expensive / low-confidence / unfamiliar	Queued for a human; LLM rationale; Discord approve/reject
Block	Over qty cap / cost ceiling / budget	Refused outright; no spend

4. System Architecture

The system is one FastAPI service with a Q-Learning core and four integrations wired around it (persistence, notifications, fulfillment, and an MCP server). The diagram below shows the full flow.



Module	Responsibility
app/main.py	FastAPI routes, request lifecycle, lifespan startup, static UI
app/agent.py	Q-Learning agent: state encoding, argmax action, confidence
app/gate.py	The dual gate — block / wait / escalate / auto routing
app/llm.py	LLM rationale layer (Anthropic/OpenAI) with deterministic fallback
app/store.py	In-memory store + budget + inventory + human-feedback loop
app/db.py	SQLite persistence (state survives a redeploy)
app/fulfill.py	Purchase-order fulfillment (PO number + optional webhook)
app/notify.py	Discord/Slack escalation notifications + approve links
app/signals.py	Demand-signal ingestion (external feed or mock) → reactive loop
mcp_server.py	MCP server exposing the live service as tools

5. The Decision Flow (end to end)

Every situation — whether from Run batch, a manual entry, an inventory re-order, or a demand signal — flows through the same pipeline in app/main.py::_process():

5.1 Encode the state

The agent discretises the situation into a 4-dimensional state. Each dimension is LOW / MEDIUM / HIGH, compared against training-time medians (config.json):

- Inventory level vs median inventory
- Demand forecast vs median forecast
- Unit price vs median price
- Remaining budget vs monthly budget

This yields $3^4 = 81$ possible states, each a key into the Q-table.

5.2 Decide the action

The agent looks up the state in the Q-table and picks the action with the highest Q-value (argmax, ε = 0 — deterministic). Actions and their order-size multipliers:

Action	Multiplier × forecast	Meaning
WAIT	0.0	Do nothing (well stocked)
BUY_SMALL	0.5	Small replenishment
BUY_MEDIUM	1.0	Standard replenishment
BUY_LARGE	2.0	Large replenishment

Confidence is computed as 0.7 × margin + 0.3 × familiarity, where margin is how decisively the top action beat the runner-up (normalised by the Q-value spread) and familiarity reflects how well-trained that state is (visit counts). An unseen state returns WAIT with zero confidence rather than guessing.

5.3 The dual gate (first match wins)

The order quantity and cost are computed, then gate.evaluate() routes the decision:

#	Condition	Route
1	quantity > MAX_ORDER_QTY, or cost > HARD_LIMIT, or cost > remaining budget	BLOCK
2	action is WAIT (zero spend)	AUTO-APPROVE
3	cost > AUTO_LIMIT	ESCALATE (cost over limit)
4	confidence < CONF_THRESHOLD	ESCALATE (low confidence)
5	visits < MIN_VISITS_FAMILIAR	ESCALATE (unfamiliar)
6	otherwise	AUTO-APPROVE

All applicable escalation reasons are recorded, even though any one triggers escalation. The confidence threshold used is the effective one (see §6).

5.4 Execute, escalate, or block

- **AUTO-APPROVE** → budget is debited, stock replenished, and a purchase order issued (§5.5). Status: AUTO_APPROVED.
- **ESCALATE** → an LLM rationale is generated, the decision is queued as PENDING, and a Discord notification is pushed (§5.6).
- **BLOCK** → the decision is recorded as BLOCKED with its reasons; no spend occurs.

5.5 Fulfillment — the agent does real work

When an order executes (auto-approved, or human-approved later), fulfill.py issues a purchase order: a PO number (e.g. PO-20261017-1A2B3C) plus a receipt. If PURCHASE_ORDER_WEBHOOK_URL is set, the receipt is POSTed downstream (supplier intake, a Google Sheet, n8n/Zapier). A webhook failure still produces a valid local PO — it never voids the order.

5.6 Escalation notifications

notify.py pushes each escalation to a Discord (or Slack-compatible) webhook as a rich card containing the rationale and two one-click links — ✅ Approve → /a/{id} and ❌ Reject → /r/{id} — so a manager can sign off a purchase order from their phone. Notifications are best-effort and never block a decision.

6. Human-in-the-Loop & the Adaptive Gate

Escalated orders wait in the approval queue. A human approves or rejects:

- **Approve** → the order executes, stock replenishes, a PO is issued, and the decision is marked HUMAN_APPROVED. Recorded as agreement with the AI.
- **Reject** → the order is voided (HUMAN_REJECTED). Recorded as an override.

Adaptive governance: a run of rejections raises the effective confidence threshold, so the gate escalates sooner (more caution); approvals relax it back toward the base. This is governance only — the learned Q-table is never modified. The dashboard shows the live confidence bar and the human-agreement rate.

7. Reactive Demand Ingestion

POST /api/ingest closes the loop from the outside world. It pulls fresh demand numbers — from an external JSON endpoint (DEMAND_SIGNAL_URL) or a built-in mock sales feed — updates the affected products' forecasts, and re-evaluates each one automatically through the full pipeline. New demand in, decisions out, with no human needed for the routine part.

8. Data Model

A Decision record (the audit unit) carries:

Field	Meaning
id, timestamp	Unique id and ISO time
product_id / name	The product
state	The 4-D encoded state
action, quantity, unit_price, cost	What the agent chose and its cost
confidence (+ margin, familiarity)	Confidence score and its breakdown
route, reasons	AUTO_APPROVED / ESCALATED / BLOCKED and why
rationale, rationale_source	The explanation text and whether LLM or fallback
status, human_decision	Lifecycle state and the human's verdict
po_number, fulfillment	Issued PO and how it was fulfilled
budget_after, notified	Budget after the order; Discord notified flag

9. API Reference

Method	Path	Purpose
GET	/api/health	Liveness (independent of model/LLM)
POST	/api/run-batch	Run the agent over the seed dataset (resets first)
POST	/api/evaluate	Evaluate one manual situation
POST	/api/ingest	Pull a demand signal and re-evaluate affected products
GET	/api/decisions	All decision records (?status= filter)
GET	/api/pending	Only pending escalations
POST	/api/decisions/{id}/approve · /reject	Human resolves an escalation
GET	/api/metrics	Dashboard summary metrics
GET	/api/inventory	Live per-product stock positions
POST	/api/inventory/{id}/reorder · /adjust	Evaluate at current stock · manual correction
GET / POST	/api/settings (+ /reset)	Read / live-edit the gate policy
GET	/api/decisions.csv	Download the audit log as CSV
GET	/a/{id} · /r/{id}	One-click approve / reject (Discord links)

10. The Application

A single-page app with a sidebar and five pages, light/dark theming, live polling, animated counters and toasts:

- **Overview** — KPI tiles, a decision-routing donut, budget & oversight, recent activity.
- **Decisions** — the audit-log feed, scenario presets, CSV export, manual entry.
- **Approvals** — the human approval queue with rationale and approve/reject.
- **Inventory** — live stock cards with coverage, status and working controls.
- **Settings** — live-edit the gate policy (persisted) and view integration status.

11. Integrations (all optional, all graceful)

Variable	Effect
DB_PATH	SQLite persistence (empty = in-memory MVP)
NOTIFY_DISCORD_WEBHOOK_URL	Push escalations to Discord with approve/reject links
PURCHASE_ORDER_WEBHOOK_URL	POST a PO receipt downstream on each executed order
PUBLIC_BASE_URL	Makes the Discord approve/reject links absolute
DEMAND_SIGNAL_URL	External demand feed for /api/ingest (else mock)
LLM_API_KEY / LLM_PROVIDER	Live LLM rationales (else deterministic template)

Every integration degrades gracefully: a missing or failing webhook never breaks a decision, voids an order, or crashes the service. The MCP server (mcp_server.py) exposes evaluate_reorder, list_pending, approve_order, reject_order, get_metrics, get_inventory and run_batch, and is kept out of the web deployment's requirements so it can never jeopardise it.

12. Technology Stack

- **Backend:** Python 3.11, FastAPI, Pydantic, httpx.
- **Agent:** Q-Learning policy over 81 discretised states (inference only).
- **Storage:** SQLite (WAL).
- **Frontend:** Vanilla HTML/CSS/JS single-page app, themeable, no framework.
- **Deploy:** Docker on Clouds by Alternate Futures ($PORT-aware).

13. Testing & Reliability

A pytest suite (24 tests) covers the dual-gate routing and precedence, agent inference and the confidence formula, the persistence round-trip, the human-feedback loop, demand ingestion, CSV export and the budget invariant. Reliability is built in: graceful model/LLM/webhook degradation, an idempotent run-batch, and state that survives a redeploy.

14. Deployment

One repo, one service. A Dockerfile builds the container; the platform injects $PORT. Environment variables (LLM key, webhooks, PUBLIC_BASE_URL, gate constants) are set in the platform dashboard — never committed. After deploy, the public URL serves the app and /api/health returns {"status":"ok"}.

15. Judging Rubric Alignment

Criterion (weight)	How ReStock AI meets it
Technical execution & reliability (25)	End-to-end working, graceful degradation, persistence, 24 passing tests
Usefulness & real-world impact (20)	Real replenishment problem, budget control, actual PO output
Theme alignment (15)	AI handles volume; human approves the risky orders (Discord)
Creativity & uniqueness (15)	Dual gate + adaptive governance + reactive ingestion; LLM explains, not decides
Automation depth & integration (15)	Discord + fulfillment webhook + MCP + demand ingestion
User experience & presentation (10)	Full app UI, light/dark, charts, live counters, toasts

16. Honest Notes on the Model

- **The Q-table is a synthesised, explainable policy** (covering all 81 states), not a trained one — and it is a drop-in artifact: a notebook-exported Q-table with the same schema plugs in unchanged.
- **The feedback loop adapts the gate, never the model.** Human overrides change the governance threshold, not the learned Q-values.
- **The LLM explains, it never decides.** Every purchasing choice is the Q-table's; the LLM only writes the rationale, with a deterministic fallback.